

状态空间搜索与回溯

怎样系统穷举、撤销并证明剪枝没有删错解

408 · 数据结构 Algorithm Extension A02

母模型：一条路径、一组候选、一个恢复合同

回溯把解空间写成决策树：节点是部分路径，边是一次选择，叶子是完整方案或失败状态。核心链条为

State Contract → Choice List → Feasibility Check → Choose → Recurse → Undo.

‘Undo’ 不是代码风格，而是返回父节点时必须恢复的状态不变量；若只 ‘push’ 不 ‘pop’，后续分支就不再代表同一棵决策树。

Owner 与停止边界

- **Owns**：决策树状态、路径/候选/结束、变体去重、剪枝证明和返回合同。
- **Uses**：DS02 的显式栈、DS04/07 的 DFS 访问；DS-A03 的可合并状态；DS-A04 的上界剪枝。
- **Stops**：树/图遍历的容器与 visited 机制回各自 Owner；若状态可安全合并，应考虑 DP，而不是盲目穷举。

目录

1	决策树与三元状态	2
1.1	可变状态的别名与恢复账本	2
2	子集、组合、排列：遍历起点就是语义	2
3	约束满足：N 皇后、数独与括号	3
4	剪枝的三种证明	4
4.1	可行性剪枝	4
4.2	最优性剪枝	4
4.3	对称性剪枝	4
5	回溯与 DP 的分流	4
5.1	DAG 中枚举所有路径：输出本身也是状态	4
6	复杂度与边界	5
7	压缩复原	5

1 决策树与三元状态

每个递归调用应能回答三件事：当前 ‘path’ 已做了什么；下一步 ‘choices’ 有哪些；何时 ‘base case’ 结束。通用骨架是：

```
1 void backtrack(State& state) {
2     if (is_complete(state)) {
3         record_copy_of_solution(state.path);
4         return;
5     }
6     for (Choice c : choices(state)) {
7         if (!feasible(state, c)) continue;
8         apply(state, c);
9         backtrack(state);
10        undo(state, c);
11    }
12 }
```

代码清单 1: 回溯状态转移骨架

‘choices(state)’ 必须由状态决定，而不是每题凭感觉改一个循环。‘record’ 要复制路径；若把可变 ‘path’ 的引用直接放入结果，回溯会把已记录方案一起清空。

正确性可用树上的覆盖与不重证明：每个合法完整方案对应根到叶的一条路径；每条路径的选择顺序唯一；去重规则只删除与另一条路径等价的分支，不删除唯一代表。复杂度必须包含输出复制：全排列有 $n!$ 个结果，每个长度 n ，时间至少 $\Omega(n \cdot n!)$ ，递归栈为 $O(n)$ 。

1.1 可变状态的别名与恢复账本

回溯状态通常包含路径、使用标记、剩余容量、计数表和棋盘等可变对象。若结果保存的是路径引用而非快照，后续 undo 会同时修改已经记录的解；若一次选择改动多个字段，撤销必须按相反顺序恢复全部字段，而不能只弹出路径元素。可以把每个选择写成一笔账：**写入哪些字段、旧值是什么、递归返回后由谁恢复、短路返回时是否仍执行恢复**。数独找到一个解后向上返回 true，每一层仍须先恢复当前尝试再决定是否继续，否则调用者复用棋盘时会得到脏状态。

“全解” “方案数” “任一可行解” “最优值” 是四种不同返回合同。全解需要在每个目标叶复制并继续；方案数只累加整数，可不保存路径；首解允许成功信号沿递归短路；最优值可以只保留当前最优和上界。把求一个解的提前 return 复制到全解任务，会静默丢失其他叶子。

2 子集、组合、排列：遍历起点就是语义

类型	状态推进	收集时机
子集	‘start’ 向后，不限制长度	每个节点都可收集
组合	‘start’ 向后，限制长度/目标	到达目标长度或目标和
排列	每层从 0 开始，‘used’ 标记元素	叶子长度达到 n

子集/组合不关心顺序，下一层只能从 ‘i+1’ 之后选，因而 ‘[1,2]’ 与 ‘[2,1]’ 只出现一次；排列关心顺序，每层重新从 0 枚举但禁止复用已用元素。这个差异不是模板记忆，而是解空间的等价关系不同。

有重复元素时先排序,使等价选择相邻;在同一递归层跳过 $i > \text{start} \ \&\& \ \text{nums}[i] == \text{nums}[i-1]$,但不同深度仍允许选择同值元素。排列去重的判据改为“前一个相同元素尚未在本路径使用”,因为同层和同路径的重复语义不同。可复选组合递归传 i 而非 $i+1$,表示下一层仍允许使用同一候选;不可复选则传 $i+1$ 。

三类问题可用“选择边是否允许回到当前下标”统一:子集/组合的下一层只能到 $i+1$ 之后,排列下一层仍扫描全候选但由 `used` 禁止同一元素身份重复,复选组合下一层允许留在 i 。因此 `start`、`used`、`i` 的差异表达的是状态空间的边界,不是三套互不相干的代码。

有重复无复选时,排序只负责让等价值相邻; $i > \text{start}$ 只在本层跳过重复,下一层仍可选相同值。例如输入 `[1,1,2]` 的子集允许 `[1,1]`,不能把所有第二个 1 都全局禁用。排列的 `!used[i-1]` 则表示同一层若前一个相同元素尚未被本路径使用,当前元素会生成重复排列;若前一个已在路径中,当前元素位于不同深度,仍可能产生唯一排列。两条条件不能互换。

目标和组合还要区分“候选可复选”和“候选数组本身有重复”:前者通过递归下标是否加一表达,后者需要排序并做同层去重。若所有候选为正数,排序后当 `candidate > remaining` 可提前终止;含零或负数时该单调剪枝失效,还可能产生无限复选,必须增加长度/次数边界。

收集时机的反例

子集在每个节点收集,组合若提前在内部节点收集会混入长度不足的方案;N 皇后要求所有解,数独通常找到一个可行解就立即返回成功。返回 `'true'` 的递归和“继续枚举所有叶子”的递归不是同一合同,必须让调用者知道是否已经短路。

3 约束满足: N 皇后、数独与括号

N 皇后按行做选择,每行选择一个列;列集合和两条对角线集合是约束状态。若当前列/对角线已占用,分支不可能恢复合法,属于可行性剪枝。用集合维护占用信息后,每个候选检查可以从扫描棋盘的 $O(n)$ 降到近似 $O(1)$,但集合修改也必须在回溯时撤销。

数独按空格填数,行、列、宫格是三个约束索引。每填一个数字后递归;若后续无解,撤销该数字并尝试下一个。选择“当前最少候选的空格”可减少搜索,但必须重新计算或维护候选集合;它是搜索顺序优化,不改变正确性。

数独的约束索引可由三个 9 位 `bitmask` 表示: `rowMask[r]`、`colMask[c]`、`boxMask[b]` 的置位代表数字已占用,候选集合是三者并集的补集。放置数字即在三个掩码置位,撤销即清位;若某空格候选数最少(MRV)为零,立即判定当前分支失败。MRV 改变的是搜索树展开顺序,不改变“每个合法填法对应一条路径”的覆盖证明;若维护候选缓存,还必须在同一行、列、宫格的相关空格上同步回滚。

N 皇后同理可用列、`row-col`、`row+col` 三个位集做近似 $O(1)$ 合法性检查;位掩码只是一种状态表示,不改变按行递归和撤销合同。棋盘关于中线镜像时,可只枚举第一行的一半列,再把解镜像补回;当 n 为奇数,中间列必须单独处理,且这种减半只适用于求全部解数量/集合、目标和约束确实保持镜像不变的情形。

合法括号把选择限制为放 `'('` 或 `')'`,状态只需 `'open,close'`: `'open < n'` 才能放左括号, `'close < open'` 才能放右括号。约束直接阻止前缀中右括号超过左括号,因此不需要生成后再检查完整字符串。卡特兰数量只描述叶子规模,不是所有递归节点数的精确上界。

括号状态也可压缩为 `balance=open-close` 和已用左括号数: `balance` 永远不负,且最终必须回到零。把右括号条件写成 `'close < n'` 是不充分的,会允许前缀先出现右括号;真正的可行性条件

是前缀余额。这个例子说明剪枝应尽早表达不可逆约束，而不是生成完整字符串后再过滤。

网格单词搜索是同一合同的另一种形状：位置是路径状态，四邻域是候选，当前路径上的 `visited` 需要做选择时标记、返回时撤销，不能像普通连通遍历一样永久标记，否则不同词路径会相互污染。IP 地址复原则把字符串切成 4 段，选择长度 1-3 的下一个字段并检查前导零、数值不超过 255；剩余字符不足/过多时可做可行性剪枝。它们共同强调：状态必须包含未来合法性所需的信息，剪枝必须依赖不可逆约束。

4 剪枝的三种证明

4.1 可行性剪枝

若当前部分路径违反不可逆约束（例如已出现同列皇后、括号余额为负、容量已超且所有后续增量非负），则所有后代都不合法，可以直接删除。证明关键是约束的单调违反：未来选择不能修复已经破坏的条件。

4.2 最优性剪枝

维护当前最好值 `'best'`，对部分路径计算乐观上界 `'upper(state)'`。若 `'upper(state) <= best'`，该分支即使做最有利选择也不能超过现有答案。上界必须乐观而非拍脑袋；低估真实可达到值会误删最优解。这个接口会被 DS-A04 的贪心上界或分支限界使用。

分支限界的状态还应区分“当前已取得值”与“剩余可取得上界”。例如装载/选取问题可用剩余容量内所有尚未选项的理想贡献构造上界；该上界可以不可实现，但不能小于真实最优延伸。求最小值时对应维护乐观下界并在下界已经不优于当前最好值时剪枝。若上界计算本身需排序或扫描，必须把它计入每个搜索节点的成本，不能只报告叶子数量。

4.3 对称性剪枝

若问题的变换把多个分支映射为等价方案，可固定一个规范代表（例如只枚举棋盘一半再恢复镜像）。必须证明目标函数和约束在该对称变换下不变；否则“减半搜索”可能漏掉非对称答案。

5 回溯与 DP 的分流

若两个不同历史到达同一状态后，未来选择集合和未来代价完全相同，则可合并为一个 DP 状态；若未来仍依赖路径细节（例如输出所有排列的顺序、已用元素集合或约束棋盘形状），贸然合并会丢解。回溯是显式遍历状态空间，DP 是证明等价后缓存状态；二者可组合，例如回溯生成选择、DP 计算剩余最优上界。

判断能否转 DP 时不能只看“递归参数相同”：必须证明两条历史在该状态之后的**可行选择集合、转移代价和目标合同**完全相同。全排列中两个不同前缀即使长度相同，`used` 集合不同，未来候选也不同，所以不能只按长度缓存；而打家劫舍若未来只依赖“前一个是否被选”就可以压缩。状态合并证明失败时，保留回溯并寻找可行性/对称/上界剪枝。

5.1 DAG 中枚举所有路径：输出本身也是状态

在有向无环图中枚举从 s 到 t 的全部路径，是回溯合同在图上的一个直接应用：当前 `path` 保存已走顶点，候选是当前顶点的出边，抵达 t 时复制一份路径并返回。递归从邻接点进入，返回后弹出末点；

由于图是 DAG，同一路径不会绕回祖先，不需要把访问标记当作全局永久状态。若题面没有 DAG 保证而允许一般图，必须改用“本路径上的 visited”防止环，而不能把一次路径搜索中的全局 visited 复用于其他路径。

正确性分两层：每条 $s \rightarrow t$ 路径的最后一条边唯一决定其父路径，因此 DFS 会覆盖且只覆盖一次；复制结果而不是保存可变 path 的引用，才能在后续 pop 后保持已输出路径。若路径数为 P 、每条平均长度为 L ，输出敏感时间至少为 $\Omega(PL)$ ，DFS 额外栈空间为 $O(|V|)$ （不计结果）；若只问路径数，可把叶子复制改为整数累加，并在 DAG 上按拓扑序做同一递推。

这一区分了三个看似相近的合同：只求是否可达可用一次 visited 的图遍历；求最短路径要维护层数或代价；求全部路径则必须保留并复制每条当前路径，不能因“访问过该顶点”就删掉另一条前缀不同的合法路径。

6 复杂度与边界

回溯最坏时间等于搜索节点数乘以每节点的状态更新/复制成本，常见为 $O(2^n)$ 、 $O(n!)$ 或约束满足的指数级；剪枝只在给出数据分布或约束条件时改善实际成本，不能把最坏界直接改写为多项式。空间至少为递归深度加状态副本；若保存所有结果，还要加输出空间。

测试时攻击：空输入、单元素、所有元素相等、重复与可复选混合、无解、唯一解、多个解、容量恰好越界、递归提前返回以及结果路径是否被后续撤销污染。对小 n 用穷举基线对拍，先验证覆盖与去重，再比较剪枝版本。

对全解问题应同时检查数量和**内容**：子集应有 2^n 个（无重复输入），排列应有 $n!$ 个，括号应符合相应卡特兰数；重复输入则用集合规范化后比较。对首解/计数题，测试递归提前返回是否执行了所有必要撤销；对位掩码优化，与朴素集合版本随机对拍，避免把掩码移位、符号扩展或清位错误误判为剪枝效果。

7 压缩复原

忘记模板时间：当前路径代表决策树哪一层？这一层的候选如何生成？选择后改变了哪些状态字段？哪些约束是不可逆的？返回父层必须恢复哪些字段？结果是全解、计数、最优值还是第一个可行解？能否给被删分支一个“绝不含目标”的证明？

一句话压缩

回溯不是“递归 + 'pop'”，而是对决策树的有合同穷举：路径定义状态，候选定义分支，约束定义合法子树，撤销恢复父状态，剪枝必须证明删除的子树不含所需答案。